Application of Machine Learning Models to Detect and Predict Alzheimer's Disease

Student Name:

Student ID:

Supervisor Name:

Acknowledgement

In this regard, I would like to thank my supervisor on the basis of their invaluable advice and support during this research. I am also grateful to the university and Kaggle for having access to the MRI dataset. The topic under dissertation would have been incomplete without the support and the motivation offered by my peers and family members.

Declaration

To the best of my knowledge, this dissertation titled Multi-Class Alzheimer's Disease MRI Classification Using Explainable Deep Learning is my original work that has not been previously submitted, in any form, to the conference for any type of academic degree or qualification at this or any other institution. All the sources, information, and input of other people have been recognized properly. I am fully accountable for the content and the conclusions made in this research project.

Abstract

Alzheimer's disease is a harmful neurodegenerative disease that affects millions of people around the world, requiring proper and early diagnosis. The current study aims at classifying the stages of Alzheimer’s disease with multi-class that includes structural MRI data following both classical machine learning methods and the deep learning technique. Decision Tree, Multilayer Perceptron (MLP), and Convolutional Neural Network (CNN) models were performed in the study and tested on a 5,000-balanced subset of MRI images. The preprocess activities involved grayscale resizing, histogram equalization, and normalization of the grayscale that gave the model a similar input. CNN-based feature extraction gave rich spatial representation of both the classical and end-to-end training CNN. To measure the discriminatory power in all classes, model analysis used confusion matrices, precision, recall, macro F1-score, ROC curves, and AUC analysis. Together with explainable AI techniques, SHAP and LIME methods were applied to interpret the predictions to point out anatomically relevant regions contributing to the classification. It can be shown that the CNN model was the most accurate (0.6576) and had the highest macro F1-score (0.642494), compared to baseline models, and was still interpretable. The results highlight that deep learning is essential in capturing minor disease patterns, yielding some clinically relevant information, and informing future creation of trustworthy, explainable diagnostic methods for Alzheimer's disease.

Table of Contents

Acknowledgement 2

Declaration 3

Abstract 4

Chapter 1: Introduction 11

1.1 Background and Rationale 11

1.2 Problem Statement 12

1.3 Significance of the Study 12

1.4 Scope and Delimitations 13

1.5 Structure of the Dissertation 13

Chapter 2: Literature Review 15

2.1 Introduction 15

2.2 Alzheimer's Disease and Clinical Assessment 15

2.2.1 Clinical Characteristics of Alzheimer's Disease 16

2.2.2 Cognitive Assessment Tools 17

2.2.3 Role of Clinical and Demographic Data 18

2.2.4 Limitations of Traditional Diagnostic Approaches 18

2.3 Machine Learning Approaches for Alzheimer's Detection 19

2.3.1 Supervised Learning Models 19

2.3.2 Deep Learning and Advanced Models 20

2.3.3 Model Evaluation and Performance Metrics 21

2.4 Explainable Artificial Intelligence in Healthcare 22

2.4.1 Explainable AI Techniques 22

2.4.2 Bias, Fairness, and Ethical Considerations 23

2.4.3 Trustworthy Deployment in Clinical Settings 24

2.5 Literature Gap 24

2.6 Conclusion 25
Chapter 3: Research Aim and Objectives 26
3.1 Research Aim 26
3.3 Research Questions 26
Chapter 4: Methods and Methodology 27
4.1 Introduction 27
4.2 Dataset Description and Source 27
4.3 Experimental Design and Data Partitioning 27
4.4 Image Preprocessing Strategy 28
4.5 Balanced Dataset Construction 28
4.6 Feature Representation and Learning Approach 29
4.7 Model Selection Rationale 29
4.8 Decision Tree Classification Framework 29
4.9 Multilayer Perceptron Architecture 30
4.10 Convolutional Neural Network Design 30
4.11 Training Configuration and Optimization 31
4.12 Evaluation Planning and Metrics 31
4.13 Explainable Artificial Intelligence Integration 31
4.14 Ethical and Practical Considerations 32
4.15 Conclusion 32
Chapter 5: Results, Implementation, and Testing 33
5.1 Introduction 33
5.2 Results 33
5.2.1 Dataset Exploration and Class Distribution Outcomes 33
5.2.2 Preprocessing and Data Transformation Results 35

5.2.3 CNN-Based Feature Extraction Results 36
5.2.4 Decision Tree Classification Results 37
5.2.5 Multilayer Perceptron Classification Results 38
5.2.6 Convolutional Neural Network End-to-End Results 38
5.2.7 Comparative Model Performance Summary 40
5.2.8 Explainable AI Results Using SHAP 40
5.2.9 Local Interpretability Results Using LIME 41
5.3 Implementation and Testing 41
5.3.1 System Architecture and Workflow 41
5.3.2 Dataset Loading and Management Implementation 42
5.3.3 Image Preprocessing Implementation Logic 45
5.3.4 Feature Extraction Implementation 48
5.3.5 Decision Tree and MLP Implementation Details 49
5.3.6 CNN End-to-End Implementation 50
5.3.7 Explainability Tool Integration 51
5.4 Conclusion 53
Chapter 6: Evaluation 54
6.1 Introduction 54
6.2 Confusion Matrix Evaluation 54
6.2.1 Decision Tree Confusion Matrix Analysis 54
6.2.2 MLP Confusion Matrix Analysis 55
6.2.3 CNN Confusion Matrix Analysis 56
6.3 Precision, Recall, and Macro F1-Score Analysis 57
6.3.1 Decision Tree Precision–Recall Performance 58
6.3.2 MLP Precision–Recall Performance 59

6.3.3 CNN Precision–Recall Performance 60
6.4 ROC Curve and AUC Evaluation.....60
6.5 Comparative Model Performance Analysis 63
6.6 SHAP-Based Interpretability Evaluation.....64
6.7 LIME-Based Local Explanation Analysis 65
6.8 Conclusion 66
Chapter 7: Conclusion.....68
7.1 Main Restrictions and Problems 68
7.2 Achievement of Aim and Objectives.....69
7.3 Achievement of Research Questions 69
7.4 Key Contributions 70
7.5 Lessons Learned (Reflection) 70
7.6 Recommendations 71
7.7 Future Work 71
7.8 Final Remark.....72
Reference.....73
Appendix 84

List of Figures

Figure 1 Alzheimer’s Disease Brain MRI Progression 16
Figure 2 explainable AI (SHAP/LIME) Interpretation for Alzheimer’s Classification 22
Figure 3 Download dataset output 34
Figure 4 Verify each class folder output.....34
Figure 5 Load image paths and labels into a data frame output 34
Figure 6 Class distribution of Alzheimer's MRI Dataset 34
Figure 7 Sample MRI image from each Alzheimer's Class.....35
Figure 8 Image preprocessing output 1 36
Figure 9 Image preprocessing output 2 36
Figure 10 CNN feature extraction output.....37
Figure 11 Decision Tree model output.....38
Figure 12 CNN classification model output.....38
Figure 13Neural network end-to-end results output 1 39
Figure 14Neural network end-to-end results output 2 39
Figure 15AI results using SHAP output.....40
Figure 16Local interpretability results using LIME output.....41
Figure 17 Install TensorFlow implementations 42
Figure 18 Kaggle dataset downloader implementations 42
Figure 19 Download dataset implementations 43
Figure 20 Verify each class folder exists implementations 44
Figure 21 load image paths and labels into a data frame implementation 44
Figure 22 Class distribution plot implementations 45
Figure 23 Image preprocessing implementations 1 46
Figure 24Image preprocessing implementations 2 47
Figure 25Image preprocessing implementations 3 47
Figure 26 Feature extraction implementations 1 48
Figure 27Feature extraction implementations 2 49
Figure 28Decision Tree and MLP Implementation.....50
Figure 29CNN End-to-End implementation 51
Figure 30 SHAP and LIME implementations 1.....52

Figure 31SHAP and LIME implementations 2 53
Figure 32Decision Tree confusion matrix analysis output 55
Figure 33MLP confusion matrix analysis output 56
Figure 34CNN confusion matrix analysis output 57
Figure 35 Model comparison precision output 57
Figure 36 Model accuracy comparison 58
Figure 37 Moel macro F1-score comparison 58
Figure 38Decision Tree confusion precision output 59
Figure 39MLP precision output 59
Figure 40CNN precision output 60
Figure 41 Decision Tree - ROC curve 61
Figure 42 MLP - ROC curve 62
Figure 43 CNN - POC curve 63
Figure 44SHAP-Based interpretability evaluation output 1 65
Figure 45SHAP-Based interpretability evaluation output 2 65
Figure 46LIME-Based local explanation analysis output 66

List of Tables

Table 1Clinical Characteristics of Alzheimer's Disease 17
Table 2Common Cognitive Assessment Tools for Alzheimer's Disease 18
Table 3Machine Learning Models Used for Alzheimer's Detection 20
Table 4Performance Metrics for Alzheimer's Prediction Models 21
Table 5Explainable AI Techniques in Healthcare 23

Chapter 1: Introduction

Alzheimer's disease is a progressive neurodegenerative disease, the major symptoms of which include memory, cognitive functioning, and reasoning performance, and consequently later cause severe dementia and dependence. It is one of the greatest public health issues in the globe, especially with the fast increase in the number of the aging population. In spite of all this, the early diagnosis of Alzheimer's disease has been challenging since the first stages are mild and could be confused with normal aging of the mind. Late diagnosis caters to the efficiency of treatment measures and imposes greater costs on healthcare systems and caregivers(Bari Antor*et al.*, 2021).

Machine learning is a field that has become a potential solution in recent years to analyze more complex medical data and aid in predicting diseases. Machine learning models may be used to determine the early signs of Alzheimer's disease by finding patterns based on cognitive testing, clinical documentation, and demographic data. The proposed paper will be devoted to the analysis and implementation of interpretable machine learning methods that can detect and predict Alzheimer's disease based on structured clinical and cognitive data and are key to the timely diagnosis and better treatment decisions in healthcare settings(Kavitha*et al.*, 2022).

1.1 Background and Rationale

The classical diagnostic tools applied in the diagnose of the Alzheimer disease are based on clinical examination and neuropsychological measures and on neuroimaging techniques such as the MMR or PET scan and so forth.Although these are useful, they can also be costly and lengthy and require specialized skills, which limits their availability in low-resource healthcare environments. Moreover, there may be differences in clinical judgment by practitioners, thus causing inconsistencies in diagnosis(Venugopalan*et al.*, 2021). These drawbacks denote the necessity of using scalable diagnostic support tools that are data-driven.

Machine learning provides a data-centric approach, as machine learning is capable of automatically analyzing a large amount of clinical and cognitive data. It has been shown that developments in supervised learning algorithms have shown high performance in classification and prediction applications in healthcare. The patterns linked to the progression of the disease

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]

		[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]

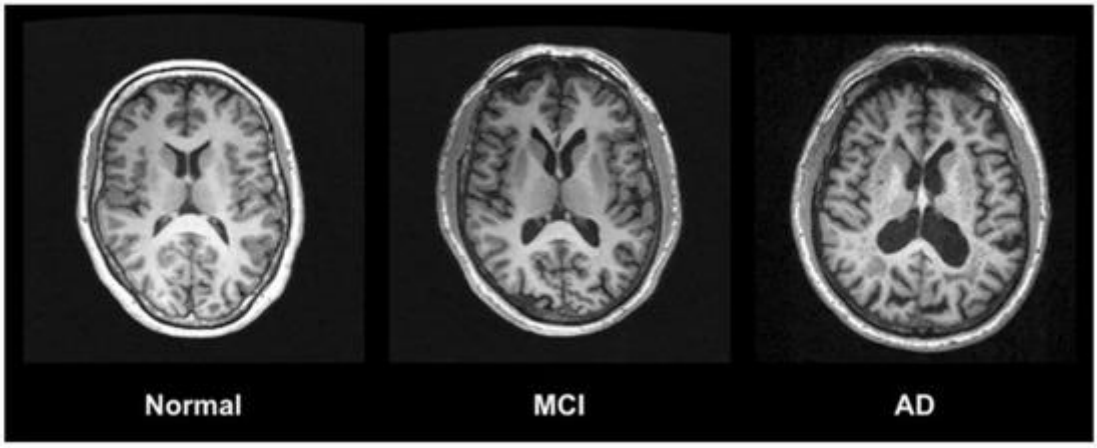
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

Clinical Feature	Description	Stage of Appearance	Diagnostic Relevance
Memory Loss	Difficulty in retaining recent information	Early	High
Cognitive Decline	Reduced reasoning and problem-solving ability	Early to Mid	High
Language Impairment	Difficulty in speech and comprehension	Mid	Moderate
Behavioral Changes	Mood swings, agitation, depression	Mid to Late	Moderate
Loss of Independence	Difficulty in daily activities	Late	High

[redacted]

[redacted]

Assessment Tool	Cognitive Domains Assessed	Score Range	Key Limitation
MMSE	Memory, attention, language	0–30	Low sensitivity in early stages
MoCA	Executive function, memory	0–30	Education bias
ADAS-Cog	Memory, language, praxis	0–70	Time-consuming
CDR	Functional performance	0–3	Subjective scoring
Clock Drawing Test	Visuospatial ability	0–5	Limited diagnostic scope

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Model	Type	Strength	Limitation
Logistic Regression	Linear classifier	High interpretability	Limited non-linearity
Support Vector Machine	Margin-based	Handles high-dimensional data	Parameter sensitivity
Random Forest	Ensemble	Captures non-linear patterns	Reduced transparency
k-Nearest Neighbors	Instance-based	Simple implementation	Computationally expensive
Naïve Bayes	Probabilistic	Fast and scalable	Assumes feature independence

[redacted]

[redacted]

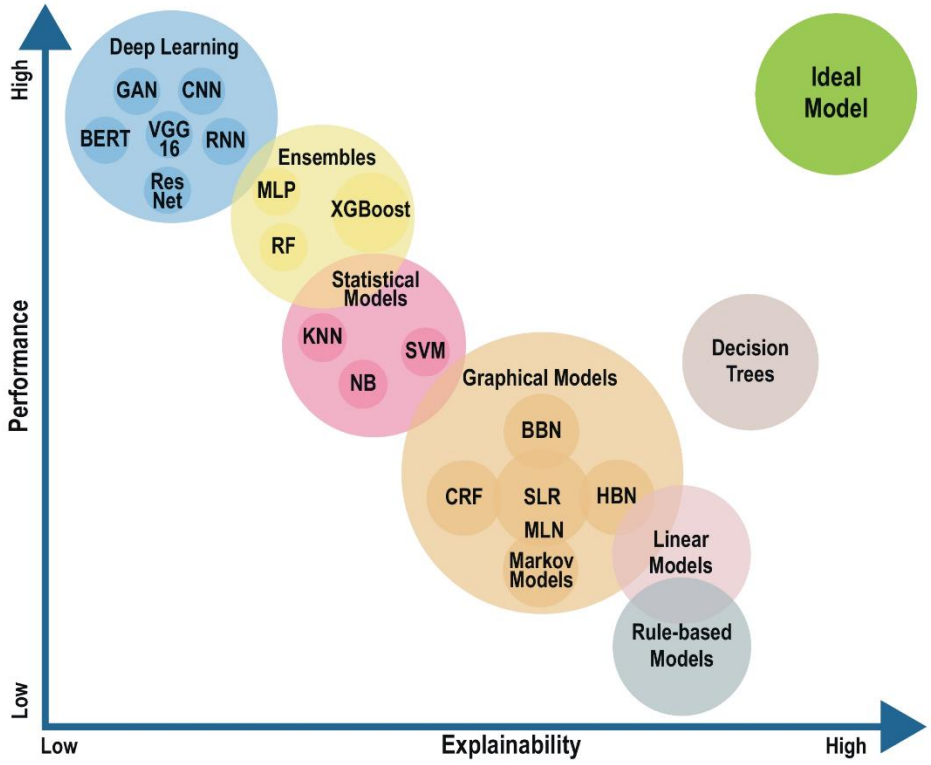
[redacted]

[redacted]

Metric	Definition	Importance	Clinical Significance
Accuracy	Correct predictions ratio	Overall performance	Misleading with imbalance
Precision	True positives among predicted positives	Reduces false alarms	Moderate
Recall (Sensitivity)	True positives detected	Critical for early diagnosis	High
F1-Score	Balance of precision and recall	Robust evaluation	High
ROC-AUC	Discrimination capability	Model comparison	High

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

XAI Method	Explanation Type	Applicable Models	Key Benefit
Feature Importance	Global	Tree-based models	Easy interpretation
LIME	Local	Model-agnostic	Case-level explanations
SHAP	Global & Local	Model-agnostic	Theoretical consistency
Partial Dependence Plot	Global	Regression & classifiers	Feature interaction insight
Rule-Based Models	Intrinsic	Decision trees	High transparency

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

- [redacted]
  - [redacted]
- [redacted]
  - [redacted]
  - [redacted]
- [redacted]
  - [redacted]
  - [redacted]
- [redacted]
  - [redacted]
- [redacted]
  - [redacted]

[redacted]

- [redacted]
  - [redacted]
- [redacted]
  - [redacted]
- [redacted]
  - [redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
Using Colab cache for faster access to the 'alzheimers-multiclass-dataset-equal-and-augmented' d
Dataset downloaded to:
/kaggle/input/alzheimers-multiclass-dataset-equal-and-augmented
```

[redacted]

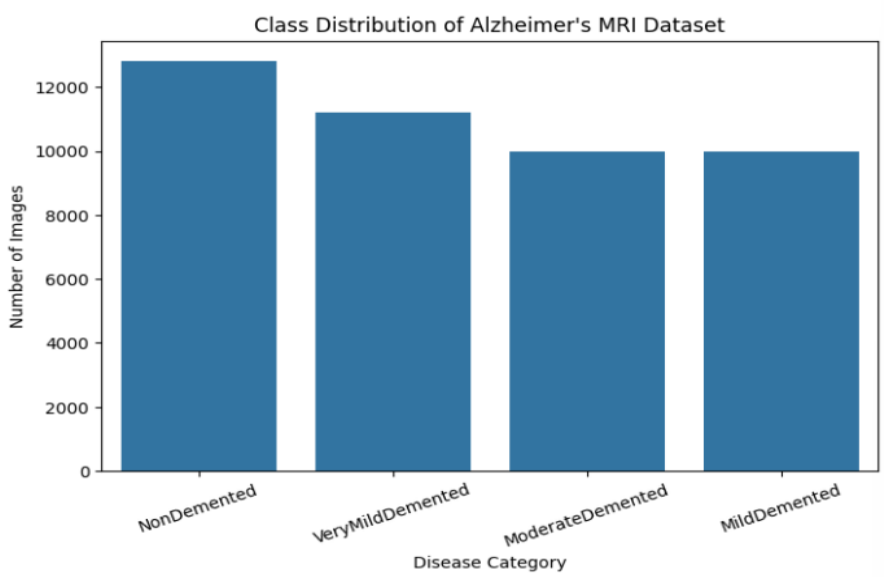
```
MildDemented -> Found
ModerateDemented -> Found
NonDemented -> Found
VeryMildDemented -> Found
```

[redacted]

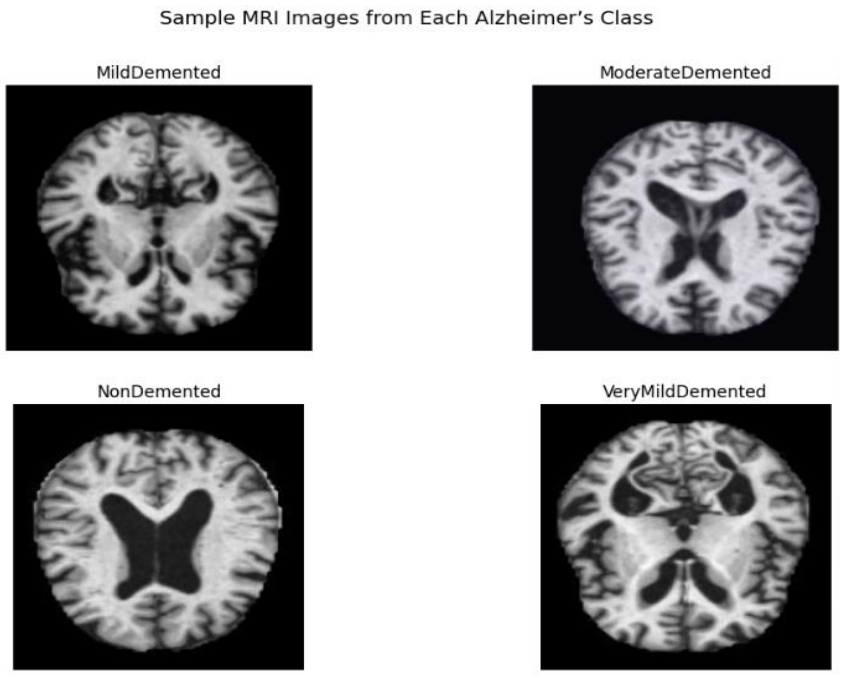
```
Total images loaded: 44000
```

	image_path	label
0	/kaggle/input/alzheimers-multiclass-dataset-eq...	MildDemented
1	/kaggle/input/alzheimers-multiclass-dataset-eq...	MildDemented
2	/kaggle/input/alzheimers-multiclass-dataset-eq...	MildDemented
3	/kaggle/input/alzheimers-multiclass-dataset-eq...	MildDemented
4	/kaggle/input/alzheimers-multiclass-dataset-eq...	MildDemented

[redacted]



[redacted]



[redacted]

[redacted]

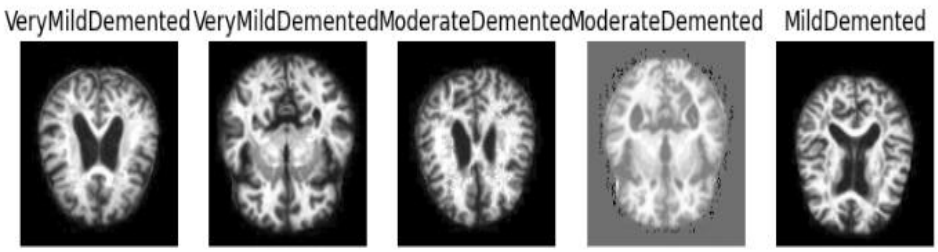
[redacted]

```
Sampling balanced dataset (5000 images total)...
Final image shape: (5000, 128, 128, 1)
Final label count: 5000

Encoded Classes:
0 -> MildDemented
1 -> ModerateDemented
2 -> NonDemented
3 -> VeryMildDemented

Train size: 2500
Test size: 2500
```

Preprocessed MRI Images (5,000 Sampled | Grayscale | Normalized)



```
Building CNN feature extractor...
Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 128, 128, 1)	0
conv2d (Conv2D)	(None, 126, 126, 32)	320
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0

```
Total params: 92,672 (362.00 KB)
Trainable params: 92,672 (362.00 KB)
Non-trainable params: 0 (0.00 B)

Extracting features from training data...
79/79 ━━━━━━━━━━━━━━━━━━━━ 27s 331ms/step
Extracting features from test data...
79/79 ━━━━━━━━━━━━━━━━━━━━ 24s 306ms/step

Feature vector shape (train): (2500, 25088)
Feature vector shape (test): (2500, 25088)
```

[redacted]

```
Training Decision Tree model...
Decision Tree Accuracy: 0.4364

Training Multilayer Perceptron (MLP)...
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer
  warnings.warn(
MLP Accuracy: 0.6388
```

[redacted]

[redacted]

[redacted]

```
Building CNN classification model...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

[redacted]

[redacted]

[redacted]

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 126, 126, 32)	320
max_pooling2d_3 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_4 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_5 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_1 (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 4)	516

Total params: 3,304,580 (12.61 MB)
Trainable params: 3,304,580 (12.61 MB)
Non-trainable params: 0 (0.00 B)

[redacted]

```
Training CNN model...
Epoch 1/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 72s 983ms/step - accuracy: 0.2595 - loss: 1.4445 - val_accuracy: 0.3840 - val_loss: 1.3610
Epoch 2/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 92s 1s/step - accuracy: 0.3387 - loss: 1.3524 - val_accuracy: 0.4560 - val_loss: 1.2537
Epoch 3/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 70s 991ms/step - accuracy: 0.4470 - loss: 1.2265 - val_accuracy: 0.5120 - val_loss: 1.1558
Epoch 4/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 70s 985ms/step - accuracy: 0.5021 - loss: 1.1022 - val_accuracy: 0.5520 - val_loss: 1.0104
Epoch 5/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 81s 977ms/step - accuracy: 0.5715 - loss: 0.9711 - val_accuracy: 0.5880 - val_loss: 0.9464
Epoch 6/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 83s 992ms/step - accuracy: 0.6075 - loss: 0.8911 - val_accuracy: 0.5560 - val_loss: 0.8871
Epoch 7/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 82s 980ms/step - accuracy: 0.6257 - loss: 0.8498 - val_accuracy: 0.6160 - val_loss: 0.8526
Epoch 8/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 70s 983ms/step - accuracy: 0.6296 - loss: 0.7910 - val_accuracy: 0.6200 - val_loss: 0.8460
Epoch 9/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 81s 977ms/step - accuracy: 0.6800 - loss: 0.7077 - val_accuracy: 0.6120 - val_loss: 0.8432
Epoch 10/10
71/71 ━━━━━━━━━━━━━━━━━━━━ 82s 975ms/step - accuracy: 0.6838 - loss: 0.7156 - val_accuracy: 0.6400 - val_loss: 0.7992

CNN Test Accuracy: 0.6576
```

[redacted]

```
Running SHAP explanations...
/usr/local/lib/python3.12/dist-packages/shap/explainers/_deep/deep_tf.py:94: UserWarning: Your TensorFlow version is newer than 2.4.0 and so graph support has been removed
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: The structure of `inputs` doesn't match the expected structure.
Expected: keras_tensor_8
Received: inputs=['Tensor(shape=(40, 128, 128, 1))']
  warnings.warn(msg)
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: The structure of `inputs` doesn't match the expected structure.
Expected: keras_tensor_8
Received: inputs=['Tensor(shape=(80, 128, 128, 1))']
  warnings.warn(msg)
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: The structure of `inputs` doesn't match the expected structure.
Expected: keras_tensor_8
Received: inputs=['Tensor(shape=(4, 128, 128, 1))']
  warnings.warn(msg)
```

```
Running LIME explanation...
100% ━━━━━━━━━━━━━━━━━━━━ 1000/1000 [00:22<00:00, 45.92it/s]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 173ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 153ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 178ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 187ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 107ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 116ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 136ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 107ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 115ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 107ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 104ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 110ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 108ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 107ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step
```

```
# Install TensorFlow
!pip install -q tensorflow
```

```
# Core utilities
import os
import pandas as pd
import numpy as np

# Image handling
from PIL import Image

# Kaggle dataset downloader
!pip install -q kagglehub
```

[redacted]

[redacted]

```python
import kagglehub

# Download dataset
dataset_path = kagglehub.dataset_download(
    "aryansinghal10/alzheimers-multiclass-dataset-equal-and-augmented"
)

print("Dataset downloaded to:")
print(dataset_path)
```

[redacted]

```python
# Main image folder
DATA_DIR = os.path.join(dataset_path, "combined_images")

# Class folders (exactly as provided)
CLASSES = [
    "MildDemented",
    "ModerateDemented",
    "NonDemented",
    "VeryMildDemented"
]

# Verify each class folder exists
for cls in CLASSES:
    class_path = os.path.join(DATA_DIR, cls)
    print(cls, "->", "Found" if os.path.exists(class_path) else "Missing")
```

[redacted]

```python
## Load image paths and labels into a DataFrame

data = []

for label in CLASSES:
    class_folder = os.path.join(DATA_DIR, label)

    for img_name in os.listdir(class_folder):
        img_path = os.path.join(class_folder, img_name)

        # Ensure valid image files only
        if img_name.lower().endswith((".jpg", ".jpeg", ".png")):
            data.append({
                "image_path": img_path,
                "label": label
            })

# Create DataFrame
df = pd.DataFrame(data)

print("Total images loaded:", len(df))
df.head()
```

[redacted]

```python
## Class Distribution Plot

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
sns.countplot(data=df, x="label", order=df["label"].value_counts().index)
plt.title("Class Distribution of Alzheimer's MRI Dataset")
plt.xlabel("Disease Category")
plt.ylabel("Number of Images")
plt.xticks(rotation=20)
plt.show()
```

[redacted]

IMAGE PREPROCESSING

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

IMG_SIZE = 128
SAMPLES_PER_CLASS = 1250  # 1250 x 4 = 5000 images

images = []
labels = []

print("Sampling balanced dataset (5000 images total)...")

for cls in CLASSES:
    class_df = df[df["label"] == cls].sample(
        SAMPLES_PER_CLASS, random_state=42
    )

    for path in class_df["image_path"]:
        img = cv2.imread(path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)      # Grayscale
        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
        img = cv2.equalizeHist(img)                      # Contrast enhancement
        img = img / 255.0                                # Normalization

        images.append(img)
        labels.append(cls)
```

[redacted]

```python
images = np.array(images)
labels = np.array(labels)

# Add channel dimension for CNN
images = images.reshape(-1, IMG_SIZE, IMG_SIZE, 1)

print("Final image shape:", images.shape)
print("Final label count:", len(labels))

# Encode labels
le = LabelEncoder()
labels_encoded = le.fit_transform(labels)

print("\nEncoded Classes:")
for i, c in enumerate(le.classes_):
    print(i, "->", c)

# 50/50 Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    images,
    labels_encoded,
    test_size=0.5,
    random_state=42,
    stratify=labels_encoded
)

print("\nTrain size:", X_train.shape[0])
print("Test size:", X_test.shape[0])
```

```python
# VISUALIZATION


plt.figure(figsize=(10,4))
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(X_train[i].squeeze(), cmap="gray")
    plt.title(le.inverse_transform([y_train[i]])[0])
    plt.axis("off")

plt.suptitle("Preprocessed MRI Images (5,000 Sampled | Grayscale | Normalized)"
plt.show()
```

[redacted]

[redacted]

FEATURE EXTRACTION (CNN-BASED)

```
# CNN FEATURE EXTRACTION

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Input

print("Building CNN feature extractor...")

input_layer = Input(shape=(128, 128, 1))

x = Conv2D(32, (3,3), activation='relu')(input_layer)
x = MaxPooling2D((2,2))(x)

x = Conv2D(64, (3,3), activation='relu')(x)
x = MaxPooling2D((2,2))(x)

x = Conv2D(128, (3,3), activation='relu')(x)
x = MaxPooling2D((2,2))(x)

x = Flatten()(x)

feature_extractor = Model(inputs=input_layer, outputs=x)

feature_extractor.summary()
```

[redacted]

```python
# EXTRACT FEATURES


print("\nExtracting features from training data...")
X_train_features = feature_extractor.predict(X_train, batch_size=32)

print("Extracting features from test data...")
X_test_features = feature_extractor.predict(X_test, batch_size=32)

print("\nFeature vector shape (train):", X_train_features.shape)
print("Feature vector shape (test):", X_test_features.shape)
```

[redacted]

MODEL DEVELOPMENT

```python
# Decision Tree + MLP
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Decision Tree Model

print("Training Decision Tree model...")

dt_model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=15,
    random_state=42
)

dt_model.fit(X_train_features, y_train)

dt_preds = dt_model.predict(X_test_features)
dt_acc = accuracy_score(y_test, dt_preds)

print("Decision Tree Accuracy:", round(dt_acc, 4))



# Multilayer Perceptron (MLP)

print("\nTraining Multilayer Perceptron (MLP)...")
```

[redacted]

[redacted]

[redacted]

```python
# CNN END-TO-END CLASSIFIER
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# One-hot encode labels for CNN
y_train_cat = to_categorical(y_train, num_classes=4)
y_test_cat = to_categorical(y_test, num_classes=4)

print("Building CNN classification model...")

cnn_model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(128,128,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(4, activation='softmax')
])

cnn_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

cnn_model.summary()

# Train CNN
print("\nTraining CNN model...")

history = cnn_model.fit(
    X_train, y_train_cat,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
```

[redacted]

```python
# SHAP + LIME (Explainable AI)
import shap
import lime
import lime.lime_image
import numpy as np
import matplotlib.pyplot as plt
from skimage.segmentation import mark_boundaries

# SHAP (CNN) – GLOBAL & LOCAL

print("Running SHAP explanations...")

# Use small background set for stability
background = X_train[np.random.choice(X_train.shape[0], 40, replace=False)]
test_samples = X_test[:4]

explainer = shap.DeepExplainer(cnn_model, background)
shap_values = explainer.shap_values(test_samples)

shap.image_plot(shap_values, test_samples)


# LIME (CNN) – LOCAL EXPLANATION

print("Running LIME explanation...")

explainer_lime = lime.lime_image.LimeImageExplainer()

# Select one image
idx = 0
image = X_test[idx].squeeze()

# LIME requires 3-channel images → replicate grayscale
image_rgb = np.stack([image, image, image], axis=-1)

def cnn_predict_lime(images):
    images = np.mean(images, axis=-1)          # Convert back to grayscale
    images = images.reshape(-1, 128, 128, 1)
    return cnn_model.predict(images)
```

[redacted]

```python
explanation = explainer_lime.explain_instance(
    image_rgb,
    cnn_predict_lime,
    top_labels=1,
    hide_color=0,
    num_samples=1000
)

predicted_class = np.argmax(cnn_model.predict(X_test[idx:idx+1]))

lime_image, mask = explanation.get_image_and_mask(
    label=predicted_class,
    positive_only=True,
    num_features=10,
    hide_rest=False
)

plt.figure(figsize=(5,5))
plt.imshow(mark_boundaries(lime_image, mask))
plt.title(f"LIME Explanation - Class: {le.inverse_transform([predicted_class])[0]}")
plt.axis("off")
plt.show()
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

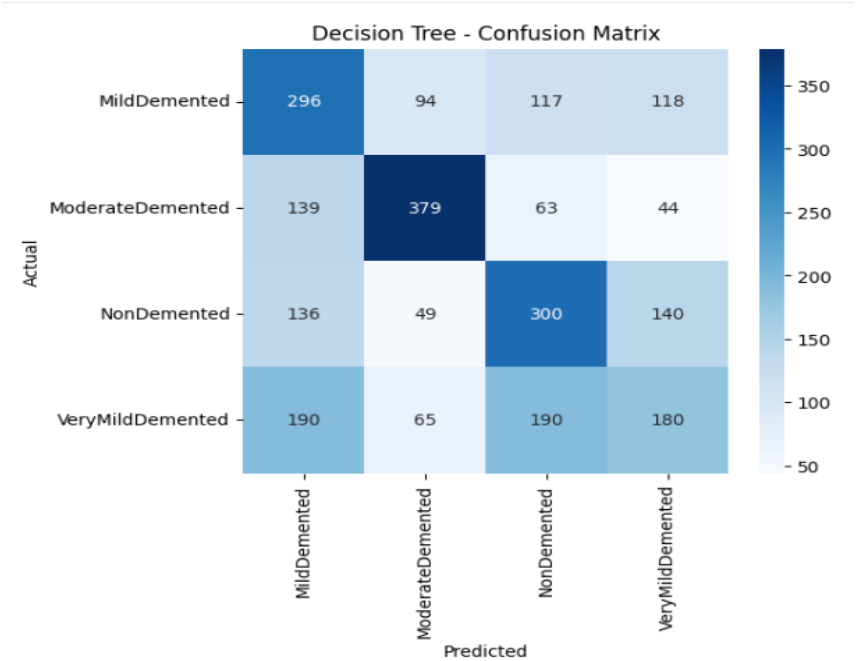
[redacted]

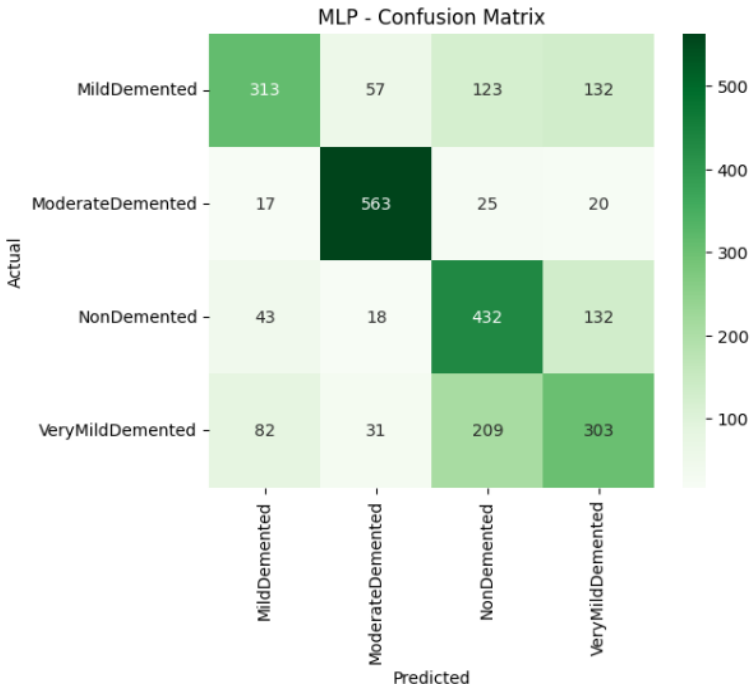
[redacted]

[redacted]

[redacted]

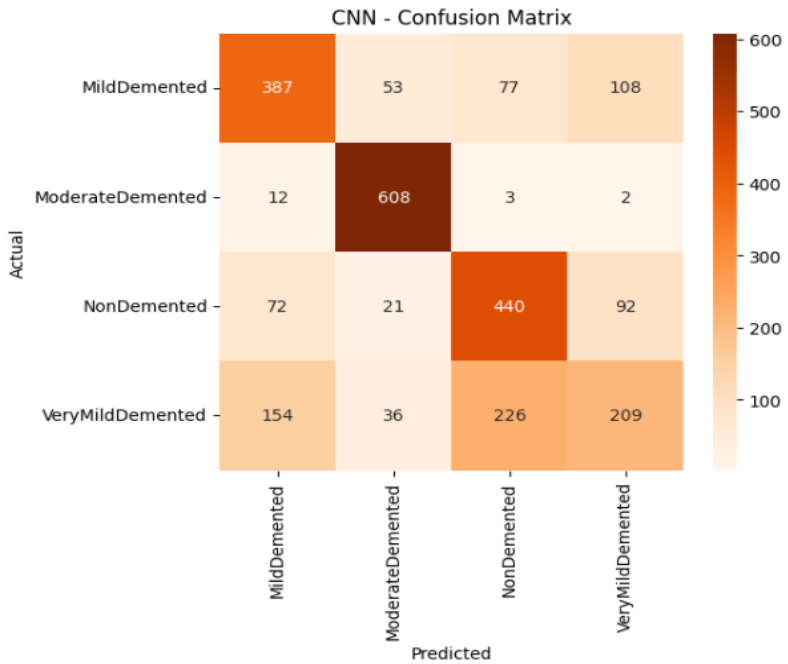
[redacted]


Decision Tree - Confusion Matrix
MildDemented
ModerateDemented
NonDemented
VeryMildDemented
Actual
296
94
117
118
139
379
63
44
136
49
300
140
190
65
190
180
350
300
250
200
150
100
50
MildDemented
ModerateDemented
NonDemented
VeryMildDemented
Predicted



[redacted]

[redacted]



[redacted]

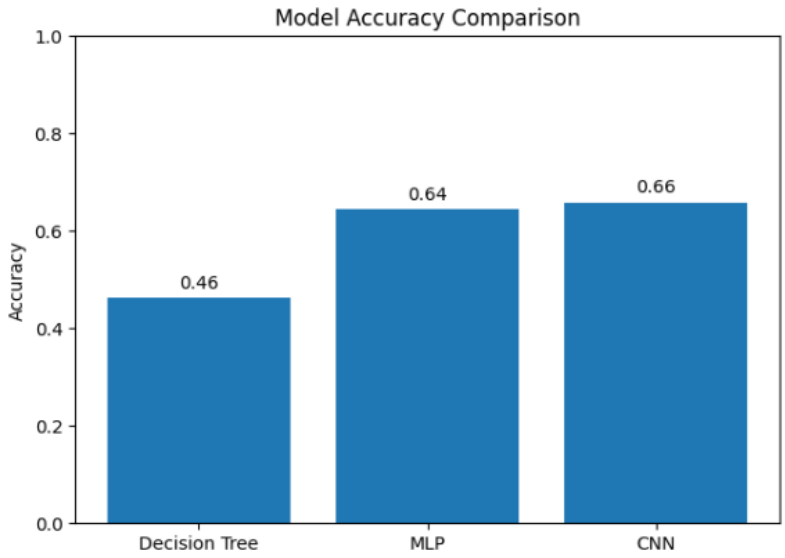
[redacted]

[redacted]

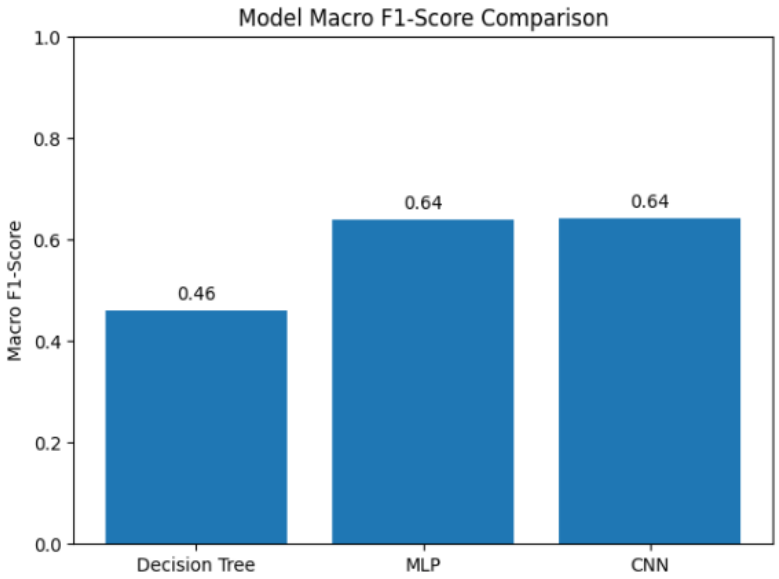
```
MODEL COMPARISON TABLE

           Model  Accuracy  Macro F1-Score
0  Decision Tree    0.4620        0.460266
1            MLP    0.6444        0.640208
2            CNN    0.6576        0.642494
```

[redacted]



[redacted]



[redacted]

[redacted]

[redacted]

DECISION TREE CLASSIFICATION REPORT

	precision	recall	f1-score	support
MildDemented	0.39	0.47	0.43	625
ModerateDemented	0.65	0.61	0.63	625
NonDemented	0.45	0.48	0.46	625
VeryMildDemented	0.37	0.29	0.33	625
accuracy			0.46	2500
macro avg	0.46	0.46	0.46	2500
weighted avg	0.46	0.46	0.46	2500

[redacted]

[redacted]

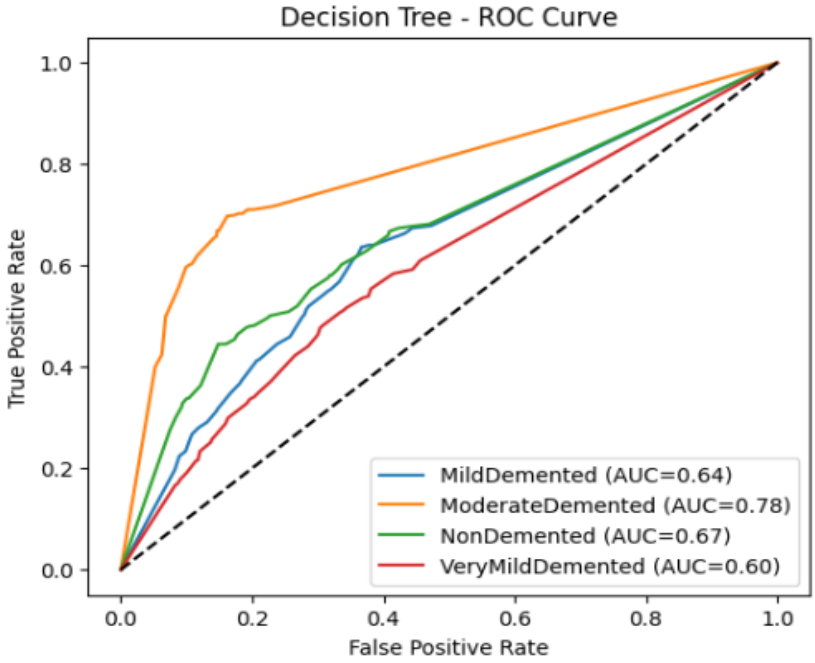
MLP CLASSIFICATION REPORT

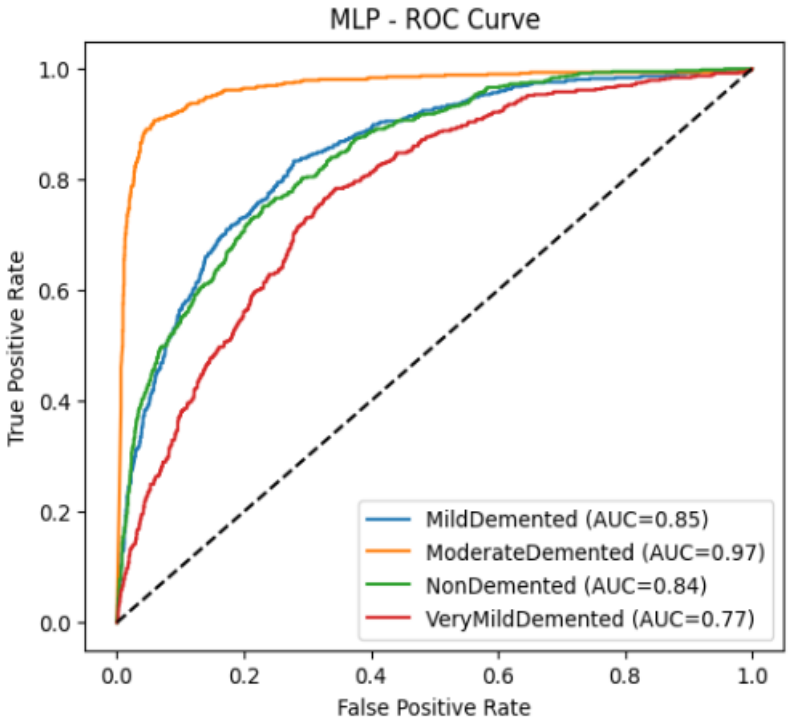
	precision	recall	f1-score	support
MildDemented	0.69	0.50	0.58	625
ModerateDemented	0.84	0.90	0.87	625
NonDemented	0.55	0.69	0.61	625
VeryMildDemented	0.52	0.48	0.50	625
accuracy			0.64	2500
macro avg	0.65	0.64	0.64	2500
weighted avg	0.65	0.64	0.64	2500

[redacted]

CNN CLASSIFICATION REPORT

	precision	recall	f1-score	support
MildDemented	0.62	0.62	0.62	625
ModerateDemented	0.85	0.97	0.91	625
NonDemented	0.59	0.70	0.64	625
VeryMildDemented	0.51	0.33	0.40	625
accuracy			0.66	2500
macro avg	0.64	0.66	0.64	2500
weighted avg	0.64	0.66	0.64	2500

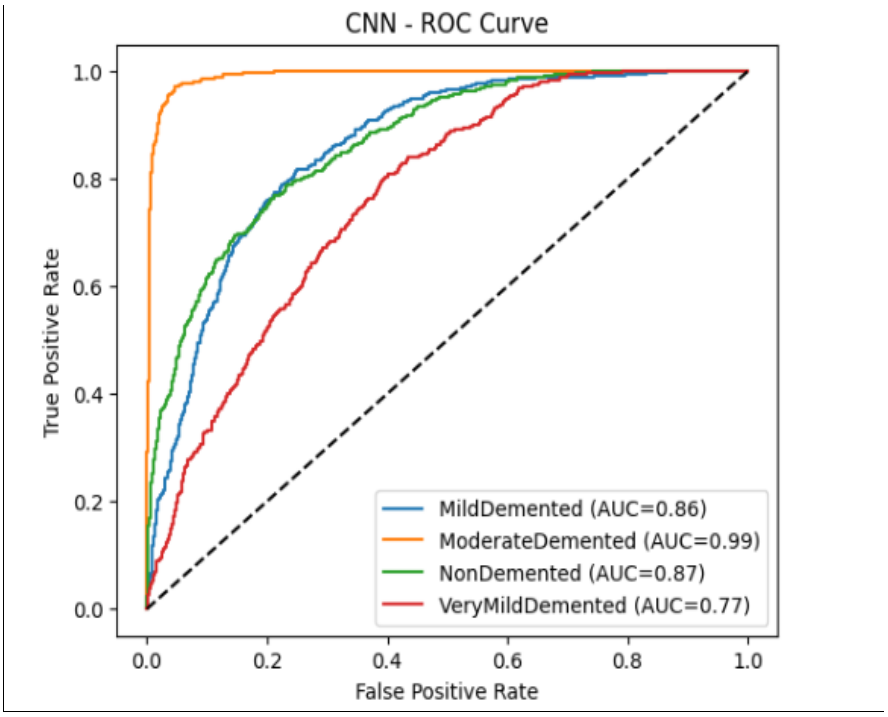

Decision Tree - ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
MildDemented (AUC=0.64)
ModerateDemented (AUC=0.78)
NonDemented (AUC=0.67)
VeryMildDemented (AUC=0.60)



[redacted]

[redacted]

[redacted]

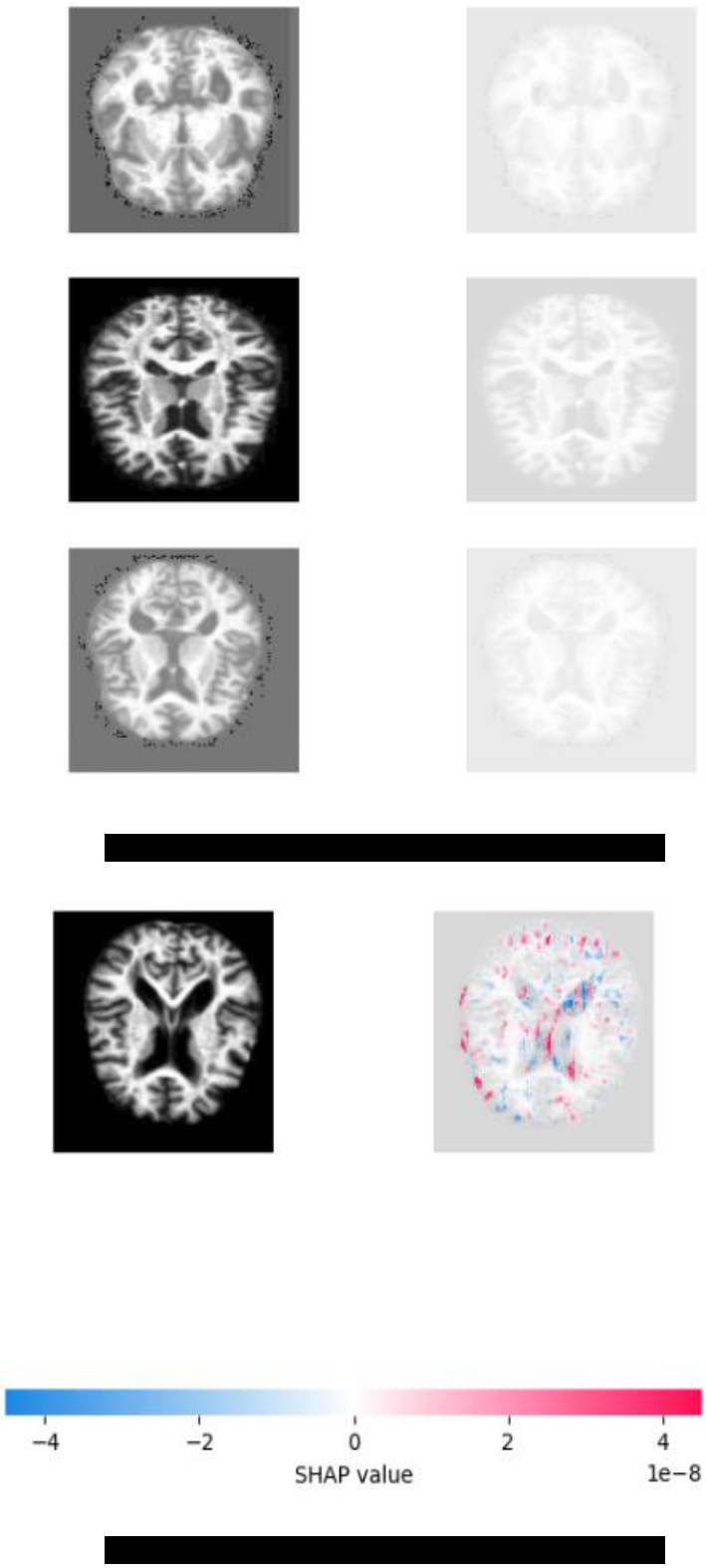

CNN - ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
MildDemented (AUC=0.86)
ModerateDemented (AUC=0.99)
NonDemented (AUC=0.87)
VeryMildDemented (AUC=0.77)

[redacted]

[redacted]

[redacted]

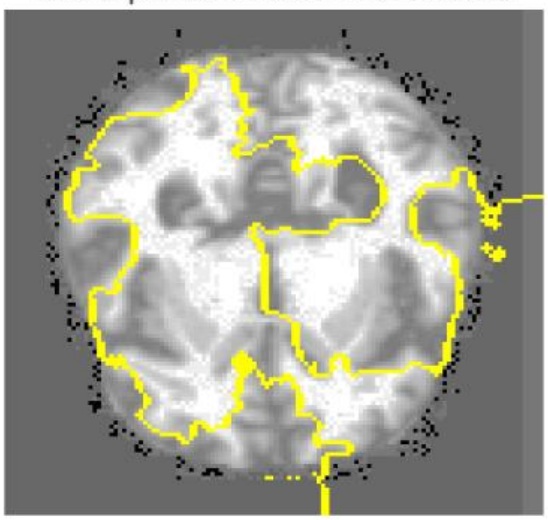
[redacted]


−4
−2
0
2
4
SHAP value
1e−8

[redacted]

[redacted]

LIME Explanation – Class: MildDemented



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]